

EventEkhane

Software Requirements Specification (SRS) and Technical Report

Event Venue Discovery and Booking Application

Document	Software Requirements Specification, v1.0 (Draft)
Project	EventEkhane (Jira key: EVT)
Prototype	Figma Make: App Onboarding and Venue Search
Team	BSCSE Team 342
Date	30 September 2026
Status	Draft for team review

Note on sources. The Figma prototype and the private Jira board could not be opened automatically while this draft was prepared. Requirements below are derived from the project name, the prototype title (onboarding and venue search) and standard practice for venue-booking applications. Items marked **[Confirm]** must be checked against the actual Figma screens and Jira backlog before submission.

Table of Contents

1. Introduction
2. Overall Description
3. System Features and Functional Requirements
4. External Interface Requirements
5. Non-Functional Requirements
6. System Architecture
7. Data Model
8. Project Management and Jira Status
9. Testing Strategy
10. Risks, Limitations and Future Work
11. Conclusion

List of Figures

- Figure 1: Use case diagram
- Figure 2: Activity diagram, onboarding and venue search
- Figure 3: Sequence diagram, booking request
- Figure 4: System architecture
- Figure 5: Entity-relationship diagram

1. Introduction

1.1 Purpose

This document specifies the software requirements for **EventEkhane**, a mobile application that helps people find and reserve venues for events such as weddings, birthday parties, corporate meetings and cultural programs. The name combines "event" with the Bengali word *ekhane* ("here"), reflecting the goal of putting every suitable venue in one place. The document is intended for developers, testers, the course supervisor and future maintainers.

1.2 Scope

EventEkhane lets a user complete a short onboarding, search venues by location, date, capacity, budget and type, view venue details, save favorites and send a booking request. Venue owners list and manage their venues, and administrators moderate content. The current milestone focuses on **app onboarding and venue search**, which is what the Figma prototype demonstrates.

1.3 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
Organizer	End user who searches for and books a venue
Venue Owner	User who lists and manages one or more venues
Admin	Platform staff who approve listings and manage users
EVT	Jira project key for EventEkhane
MVP	Minimum viable product, the first releasable version
API	Application Programming Interface

1.4 Problem Statement

Finding an event venue in a city like Dhaka is usually done through phone calls, social media groups and word of mouth. Prices, availability and capacity are hard to compare, and organizers lose time contacting venues that turn out to be unsuitable. EventEkhane addresses this with a searchable, filterable catalogue and a direct booking-request flow.

1.5 Objectives

- Reduce the time needed to shortlist venues to a few minutes.
- Give venue owners a simple channel to reach organizers.
- Provide a clear, guided first-run experience (onboarding) that sets location and event preferences.
- Deliver a maintainable, well-documented system suitable for iterative Agile development.

2. Overall Description

2.1 Product Perspective

EventEkhane is a new, self-contained system with a cross-platform mobile client and a cloud backend. It integrates with third-party services for maps, authentication (phone/email/Google), push notifications and image storage. [Confirm final tech choices]

2.2 User Classes

User class	Description	Key needs
Organizer	General public planning an event	Fast search, filters, transparent price, saved venues, booking requests
Venue Owner	Hall, community center, hotel or resort manager	Listing management, availability calendar, request handling
Admin	Platform operator	Approve listings, handle reports, view analytics

2.3 Operating Environment

- Client: Android (primary) and iOS via a cross-platform framework; Android 8.0+ and iOS 14+.
- Backend: cloud-hosted REST API with a managed database.
- Network: works on 3G/4G/Wi-Fi; graceful behavior when offline.

2.4 Design and Implementation Constraints

- Academic project timeline and a small team; scope is limited to the MVP.
- Must use free or low-cost cloud tiers.
- UI must follow the approved Figma design system.

2.5 Assumptions and Dependencies

- Venue data is entered by owners or seeded manually for the pilot.
- Users have a smartphone with internet and location services.
- Online payment is out of scope for the MVP; payment happens offline between organizer and owner.

3. System Features and Functional Requirements

3.1 Feature Overview

ID	Feature	Priority	Milestone
F1	App onboarding	High	MVP
F2	Account registration and login	High	MVP
F3	Venue search and filters	High	MVP
F4	Venue details	High	MVP
F5	Favorites	Medium	MVP
F6	Booking request	High	MVP+
F7	Venue owner dashboard	Medium	Phase 2
F8	Admin moderation	Medium	Phase 2
F9	Notifications, reviews	Low	Phase 3

3.2 F1: App Onboarding

On first launch the app introduces its value through a short swipeable walkthrough, then collects the user's basic preferences so that search results are relevant from the start.

Req. ID	Requirement
FR-1.1	The system shall show a splash screen followed by 3 introductory onboarding slides on first launch.
FR-1.2	The user shall be able to skip or move through slides using Next, Back and Skip controls.
FR-1.3	The system shall request location permission and explain why it is needed; the app shall remain usable if permission is denied by allowing manual city or area selection.
FR-1.4	The system shall let the user select event types of interest (wedding, birthday, corporate, etc.).
FR-1.5	The system shall not show onboarding again after completion unless the app data is cleared.

3.3 F2: Registration and Login

Req. ID	Requirement
FR-2.1	The system shall allow registration with mobile number (OTP) or email and password. [Confirm]
FR-2.2	The system shall allow guest browsing of venues; login is required only to save favorites or send a booking request.
FR-2.3	The system shall let users reset a forgotten password.
FR-2.4	The system shall let users choose a role (Organizer or Venue Owner) at sign-up.

3.4 F3: Venue Search and Filters

Venue search is the core feature of the current prototype. Users can search by keyword, then refine results with filters and sorting.

Req. ID	Requirement
FR-3.1	The system shall provide a search bar accepting venue name, area or keyword, with recent-search suggestions.

Req. ID	Requirement
FR-3.2	The system shall support filters: location/area, event type, date availability, guest capacity, price range, venue type (community center, hotel, resort, rooftop) and amenities (AC, parking, catering, stage).
FR-3.3	The system shall support sorting by relevance, price (low to high, high to low), rating and distance.
FR-3.4	Results shall be displayed as cards showing photo, name, area, capacity, starting price and rating.
FR-3.5	The system shall show a clear empty state with a prompt to relax filters when no result matches.
FR-3.6	The system shall load results in pages (infinite scroll) of 10 to 20 items.
FR-3.7	The system shall offer a map view of results. [Confirm if in prototype]

3.5 F4 and F5: Venue Details and Favorites

Req. ID	Requirement
FR-4.1	The system shall display a photo gallery, description, address with map, capacity, pricing, amenities, rules and contact options.
FR-4.2	The system shall show an availability calendar indicating booked and open dates.
FR-5.1	A logged-in user shall be able to add or remove a venue from Favorites with one tap.
FR-5.2	The system shall list all favorites in a dedicated screen.

3.6 F6: Booking Request

Req. ID	Requirement
FR-6.1	The user shall be able to submit a request with date, time slot, guest count, event type and notes.
FR-6.2	The system shall notify the venue owner and store the request with status Pending.
FR-6.3	The owner shall be able to Accept, Decline or Propose another date; the organizer shall be notified of the outcome.
FR-6.4	The system shall prevent double confirmation of the same venue, date and slot.

3.7 F7 and F8: Owner Dashboard and Admin Moderation (Phase 2)

- Owners create, edit and unpublish listings, upload photos, set prices and manage the availability calendar.
- Admins approve or reject new listings, suspend accounts, review reported content and view usage statistics.

3.8 Sample Use Case: Search and Shortlist a Venue

Item	Description
Actor	Organizer
Precondition	App installed; onboarding completed
Main flow	1. Open Search. 2. Enter area or keyword. 3. Apply filters (date, capacity, budget). 4. Review result cards. 5. Open a venue. 6. Tap Favorite or Request Booking.
Alternate flow	No results: system shows suggestions to widen filters. Location denied: user selects area manually.
Postcondition	Venue saved to favorites or request created with status Pending

3.9 Use Case Diagram

Figure 1 shows the actors and the main use cases of EventEkthane. Search Venues is extended by Apply Filters.

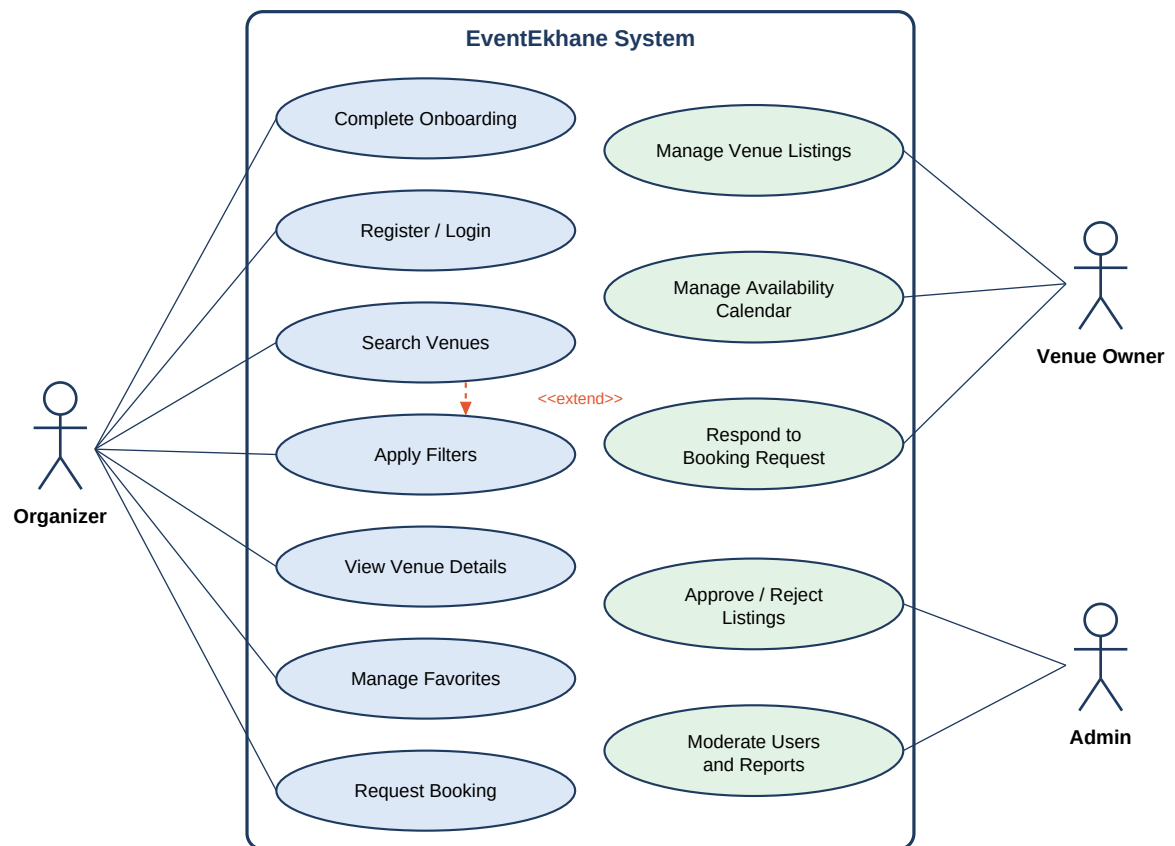


Figure 1: Use case diagram of EventEkthane

3.10 Onboarding and Venue Search Flow

Figure 2 is the activity flow from first launch through onboarding to viewing a venue, matching the Figma prototype scope. [Confirm against prototype]

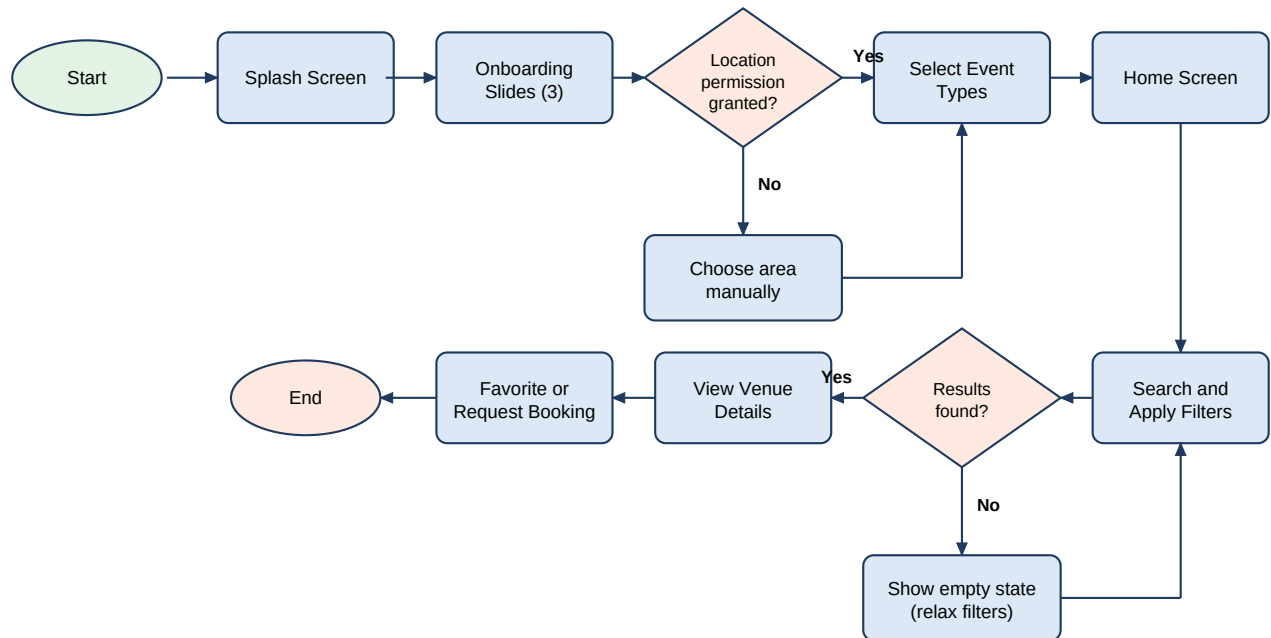


Figure 2: Activity diagram, onboarding and venue search

3.11 Sequence Diagram: Booking Request

Figure 3 shows how a booking request travels through the system and how the owner's decision returns to the organizer.

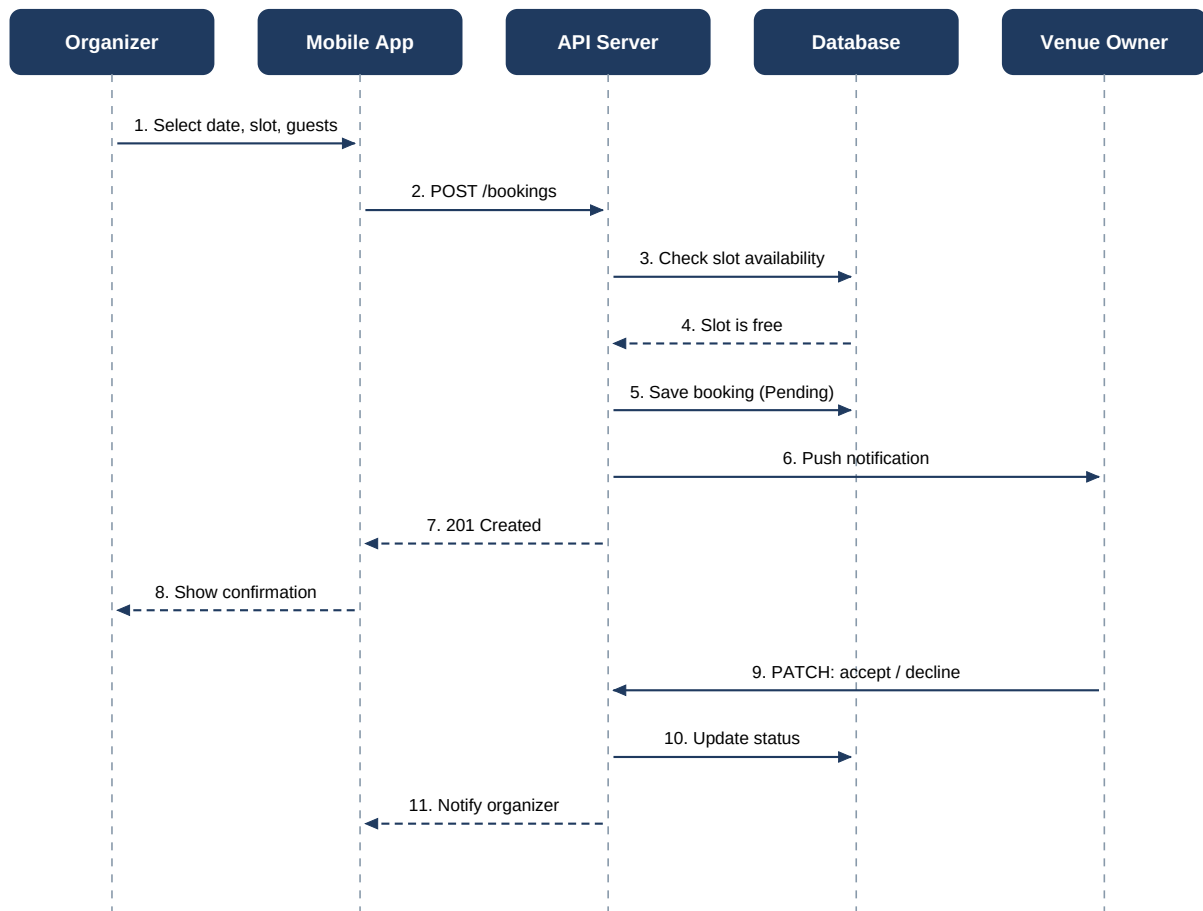


Figure 3: Sequence diagram of the booking request

4. External Interface Requirements

4.1 User Interface

The UI follows the Figma Make prototype *App Onboarding and Venue Search*. Key screens are listed below; [Confirm] screen names against the prototype.

Screen	Purpose	Main elements
Splash	Brand introduction	Logo, loading indicator
Onboarding 1-3	Explain value	Illustration, headline, Next/Skip, progress dots
Permissions and Preferences	Location and event types	Permission prompt, chips for event types
Login / Sign-up	Authenticate	Phone or email fields, OTP, social login
Home	Entry point	Search bar, categories, featured venues
Search Results	Browse	Filter bar, sort, venue cards
Filter Sheet	Refine	Sliders, checkboxes, date picker
Venue Details	Decide	Gallery, info, calendar, CTA button
Favorites	Shortlist	Saved venue list

UI principles: consistent spacing and typography, touch targets of at least 44 px, readable contrast, and support for English with Bangla localization as a planned enhancement.

4.2 Hardware and Software Interfaces

- GPS/location services for nearby venues.
- Camera and gallery access for owners uploading photos.
- Maps SDK for location display and directions.
- SMS/OTP provider and push notification service.

4.3 Communication Interfaces

The client communicates with the backend over HTTPS using JSON REST APIs. Authentication uses token-based sessions (JWT). Push notifications use the platform notification service.

4.4 Main API Endpoints (proposed)

Method	Endpoint	Description
POST	/auth/register, /auth/login	Create account, sign in
GET	/venues?q=&area;=&capacity;=&price;_min=&price;_max=&date;=	Search with filters
GET	/venues/{id}	Venue details and availability
POST	/favorites, DELETE /favorites/{id}	Manage favorites
POST	/bookings	Submit booking request
PATCH	/bookings/{id}	Owner updates status

5. Non-Functional Requirements

Category	Requirement
Performance	Search results shall appear within 2 seconds on a typical 4G connection for up to 1,000 venues; app cold start under 3 seconds.
Scalability	Backend shall support at least 500 concurrent users in the pilot and scale horizontally later.
Availability	Target 99% uptime during the pilot period.
Security	All traffic over HTTPS; passwords hashed; OTP rate limiting; role-based access control; input validation against injection.
Privacy	Location used only with permission; personal data limited to what is needed; users can delete their account.
Usability	A first-time user should be able to find and open a venue within 3 minutes without instructions.
Reliability	Failed network calls show retry options; drafts of booking forms are not lost on interruption.
Compatibility	Android 8.0+, iOS 14+, screen sizes from 5 to 7 inches.
Maintainability	Modular code, coding standards, code review through pull requests, and documented APIs.
Accessibility	Scalable text, sufficient color contrast, and screen-reader labels for key controls.

6. System Architecture

EventEkhane uses a three-tier architecture: a mobile presentation layer, an application/API layer, and a data layer. [Confirm final stack]

Layer	Responsibility	Proposed technology
Presentation	Screens, navigation, state, form validation	React Native or Flutter (prototype built in Figma Make/React)
Application	Business logic, authentication, search, booking rules	Node.js/Express or Firebase Cloud Functions
Data	Persistent storage of users, venues, bookings	PostgreSQL or Firestore; object storage for images
External	Maps, OTP/SMS, push notifications	Google Maps, Firebase Auth/FCM

The search module applies filters as indexed database queries and returns paginated results. Images are served through a CDN with thumbnails to keep list scrolling smooth. Booking creation runs in a transaction that checks slot availability to avoid double booking.

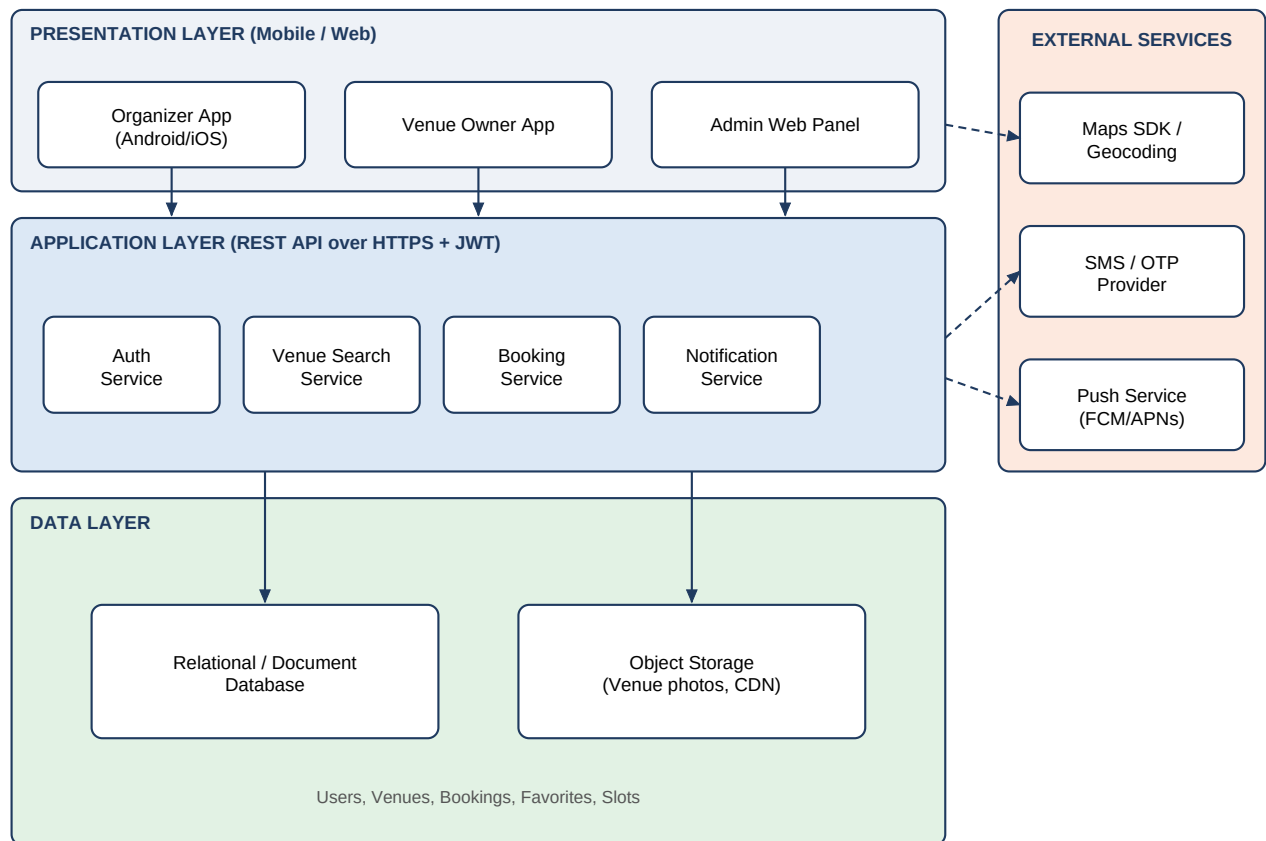


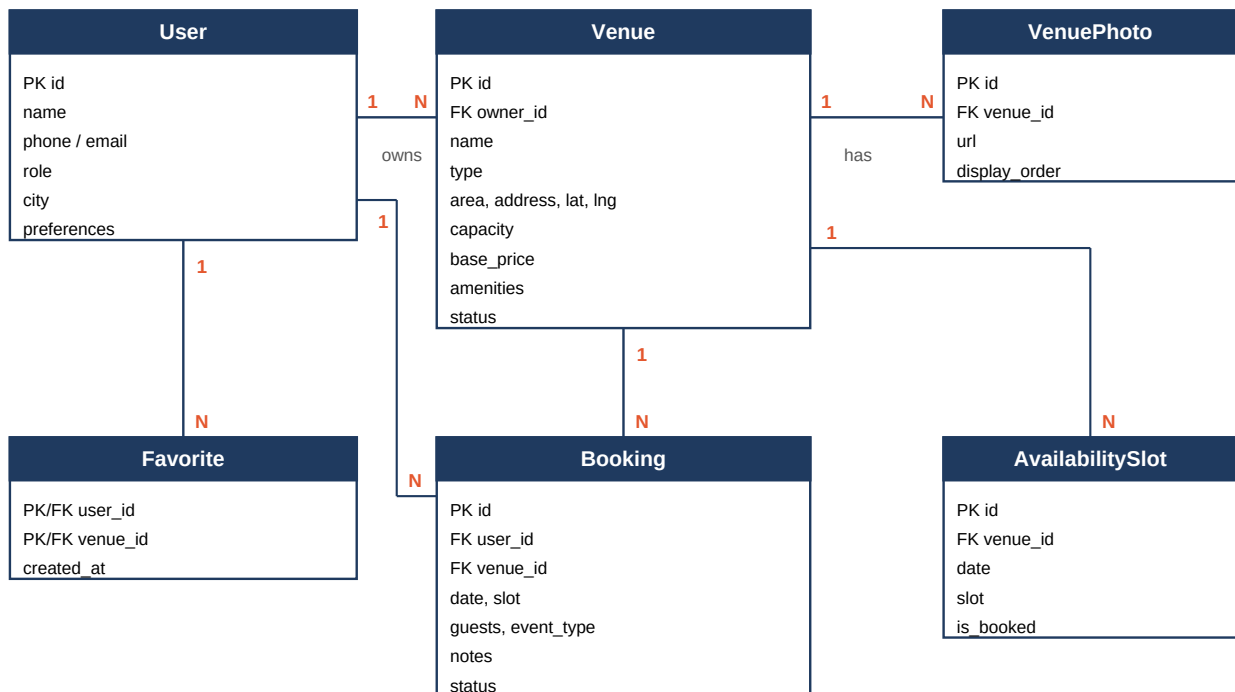
Figure 4: System architecture of EventEkhane

7. Data Model

The core entities and their main attributes are shown below.

Entity	Key attributes	Relationships
User	id, name, phone, email, role, city, preferences	1 User (owner) has many Venues; 1 User has many Bookings and Favorites
Venue	id, owner_id, name, type, description, area, address, lat, lng, capacity, base_price, amenities, status	Has many Photos, AvailabilitySlots, Bookings
VenuePhoto	id, venue_id, url, order	Belongs to Venue
AvailabilitySlot	id, venue_id, date, slot, is_booked	Belongs to Venue
Booking	id, venue_id, user_id, date, slot, guests, event_type, notes, status, created_at	Belongs to User and Venue
Favorite	user_id, venue_id, created_at	Links User and Venue (many-to-many)
Review (Phase 3)	id, venue_id, user_id, rating, comment	Belongs to Venue and User

Booking status values: **Pending, Accepted, Declined, Cancelled, Completed**. Venue status values: **Draft, PendingApproval, Published, Suspended**.



Crow's-foot style notation simplified: 1 = one, N = many. Favorite is the join table between User and Venue (many-to-many).

Figure 5: Entity-relationship diagram

8. Project Management and Jira Status

The team follows Agile Scrum using Jira (project key **EVT**, board 34). Work is organized into epics, broken into user stories and tasks, and planned in sprints. The table below maps the epics to the requirements in this document.

Epic (proposed)	Related requirements	Sprint	Status
EVT: Onboarding and permissions	FR-1.x	Sprint 1	[Update from Jira]
EVT: Authentication	FR-2.x	Sprint 1-2	[Update from Jira]
EVT: Venue search and filters	FR-3.x	Sprint 2	[Update from Jira]
EVT: Venue details and favorites	FR-4.x, FR-5.x	Sprint 3	[Update from Jira]
EVT: Booking request	FR-6.x	Sprint 3-4	[Update from Jira]
EVT: Testing and documentation	All	Sprint 4	[Update from Jira]

Definition of Done: code reviewed and merged, unit tests passing, acceptance criteria verified against the Figma design, and documentation updated.

Please replace the placeholder status column with the real ticket keys (EVT-n), assignees and story points from the backlog.

9. Testing Strategy

Level	Focus	Tools (proposed)
Unit	Filter logic, validation, price and date utilities	Jest or Flutter test
Integration	API and database, booking transaction	Postman, Supertest
UI / Acceptance	Onboarding flow, search and filter scenarios vs Figma	Manual test cases, Detox or integration_test
Usability	Task completion with 5 to 8 real users	Observation, SUS questionnaire
Performance	Search response time with sample data	k6 or JMeter

Sample test cases

ID	Scenario	Expected result
TC-01	Complete onboarding and deny location permission	User reaches Home and can pick an area manually
TC-02	Search "Banani" with capacity 300 and budget filter	Only matching venues shown, sorted by chosen order
TC-03	Apply filters producing zero matches	Empty state with suggestion to relax filters
TC-04	Guest taps Favorite	Login prompt appears; venue saved after login
TC-05	Two users request the same slot	Second confirmation is rejected

10. Risks, Limitations and Future Work

10.1 Risks

Risk	Impact	Mitigation
Insufficient venue data at launch	Search feels empty	Seed data manually; partner with a few venues
Scope creep beyond MVP	Missed deadline	Prioritize in Jira; freeze scope per sprint
Third-party service cost or limits	Feature disabled	Use free tiers; abstract service layer
Inaccurate availability	Double booking, distrust	Owner-managed calendar; server-side checks
Small team, knowledge silos	Delays	Pair work, code review, documentation

10.2 Limitations

- No in-app payment or contract handling in the MVP.
- Availability depends on owners keeping calendars updated.
- English-only interface initially.

10.3 Future Work

- Online payment (bKash, Nagad, cards) and advance deposit.
- Bangla localization and voice search.
- Reviews and ratings, verified bookings.
- Price comparison, recommendations based on past searches.

- Add-on services such as catering, decoration and photography.

11. Conclusion

EventEkhane addresses a practical problem by making venue discovery fast and transparent. This SRS defines the scope of the MVP, centered on onboarding and venue search, and sets out functional and non-functional requirements, architecture, data model, testing and project tracking. Once the placeholders marked [Confirm] and [Update from Jira] are aligned with the Figma prototype and the Jira backlog, the document can serve as the baseline for development and evaluation.